Автономная некоммерческая организация высшего образования
«Московский университет «СИНЕРГИЯ»

Факультет информационных технологий
Направление подготовки: Информационные системы и технологии
Дисциплина: Основы методологии DevOps

ОБЗОРНАЯ РЕФЕРАТИВНАЯ РАБОТА

на тему:

«GitOps: технологии Git, GitHub и GitLab в современной разработке программного обеспечения»

Выполнил(а): Лоор Идальго Жан Пьерр
Группа: ДБО-301рпо
Проверил: _____

Москва, 2026

Аннотация

В работе рассматриваются технологии Git, GitHub и GitLab как основа современного GitOps-подхода и практик CI/CD. Особое внимание уделено роли Git как источника истины, организации командной разработки через pull request и merge request, автоматизации сборки, тестирования и доставки программного обеспечения. Работа содержит сравнительный анализ GitHub и GitLab, пример практических pipeline-конфигураций, таблицы и схему GitOps-процесса.

Ключевые слова: Git, GitHub, GitLab, GitOps, CI/CD, DevOps, репозиторий, pipeline, merge request, pull request, контроль версий.

Содержание

Введение

1. Понятие GitOps и место Git в DevOps-процессах
2. Git как распределенная система контроля версий
3. GitHub: совместная разработка и автоматизация workflows
4. GitLab: единая платформа для репозиториев и CI/CD
5. Сравнение GitHub и GitLab в контексте GitOps
6. Практическая организация GitOps-процесса
7. Безопасность, контроль качества и типичные ошибки

Заключение

Список использованных источников

Введение

Современная разработка программного обеспечения стала непрерывным процессом, в котором код, инфраструктура, тестирование и развертывание тесно связаны между собой. Если раньше команда могла долго разрабатывать систему локально и только затем передавать ее на интеграцию, то сегодня такой подход считается рискованным: ошибки обнаруживаются поздно, сборка становится непредсказуемой, а выпуск новой версии занимает много времени.

На смену разрозненным ручным действиям пришли практики DevOps, CI/CD и GitOps. Их общая идея состоит в том, чтобы хранить изменения в системе контроля версий, автоматически проверять каждое изменение и делать состояние программного продукта прозрачным для всей команды. В материалах к работе CI/CD представлен как непрерывный цикл, включающий этапы code, build, test, release, deploy, operate и monitor, а среди ключевых инструментов показаны GitHub, GitLab, Jenkins, Docker, Kubernetes, Terraform, Prometheus и Argo CD [10].

Тема GitOps особенно актуальна, потому что Git в этом подходе используется не только как хранилище исходного кода, но и как источник истины для конфигураций, pipeline-файлов, инфраструктурных описаний и процесса доставки. OpenGitOps рассматривает GitOps как вендорно-нейтральный подход, основанный на принципах совместимости инструментов, повторяемости и управляемости операционных процессов [8].

Цель работы - рассмотреть технологии Git, GitHub и GitLab в контексте GitOps и показать, как они применяются для организации современной командной разработки, автоматизированной сборки, тестирования и доставки программного обеспечения.

Для достижения цели поставлены следующие задачи:

- раскрыть понятие GitOps и его связь с DevOps и CI/CD;

- описать роль Git как распределенной системы контроля версий;
- рассмотреть возможности GitHub для командной разработки и автоматизации workflows;
- проанализировать GitLab как платформу для репозиториев, merge request и CI/CD;
- сравнить GitHub и GitLab по ключевым критериям;
- привести практический пример GitOps-процесса и pipeline-конфигураций;
- выделить риски, типичные ошибки и требования безопасности.

Объект исследования - процессы командной разработки программного обеспечения. Предмет исследования - технологии Git, GitHub и GitLab как инструменты GitOps и CI/CD.

1. Понятие GitOps и место Git в DevOps-процессах

GitOps - это подход к управлению разработкой, доставкой и эксплуатацией программного обеспечения, при котором Git-репозиторий становится центральным источником истины. В Git фиксируются не только изменения исходного кода, но и конфигурации сборки, тестирования, инфраструктуры и развертывания. Любое изменение должно проходить через коммит, review и автоматическую проверку.

В классической DevOps-модели важны автоматизация, быстрые циклы обратной связи и совместная ответственность команды. GitOps конкретизирует эти принципы: все изменения описываются декларативно, версионируются в Git, проверяются в pipeline и применяются автоматизированными средствами. Благодаря этому команда получает воспроизводимый и контролируемый процесс.

Важная особенность GitOps - отказ от ручных неотслеживаемых изменений. Если конфигурация сервера или приложения меняется вручную, команда теряет прозрачность: сложно понять, кто внес изменение, почему оно произошло и как вернуть систему к предыдущему состоянию. В GitOps каждое изменение имеет автора, комментарий, историю и возможность отката.

В рамках темы лабораторного задания GitOps связан прежде всего с технологиями Git, GitHub и GitLab. Git обеспечивает локальный и удаленный контроль версий. GitHub и GitLab предоставляют командные платформы для хранения репозиториев, code review, задач, pull/merge request, CI/CD, секретов, артефактов и интеграций.

GitOps нельзя сводить только к Kubernetes или Argo CD, хотя такие инструменты часто используются в промышленной практике. Для учебной обзорной работы важно понять основу: Git является единым местом фиксации

изменений, GitHub и GitLab организуют совместную работу, а pipeline автоматически проверяет и доставляет изменения.

Рисунок 1 - Общая логика GitOps-процесса: от изменения в Git до проверки и доставки.

Git	**→**	**GitHub / GitLab**	**→**	**CI pipeline**	**→**
коммиты, ветки, теги		review, pull/merge request		сборка, тесты, артефакты	GitOps CD → Production → Monitor

В данной схеме Git фиксирует изменения, GitHub/GitLab обеспечивают review и совместную работу, CI pipeline выполняет сборку и тестирование, а GitOps CD применяет проверенное состояние в целевой среде.

2. Git как распределенная система контроля версий

Git - это распределенная система контроля версий, предназначенная для хранения истории изменений файлов и совместной работы над проектами. Официальная документация характеризует Git как быструю, масштабируемую распределенную систему контроля версий с богатым набором команд [1]. В отличие от простого хранения файлов в папках, Git фиксирует каждое значимое изменение в виде коммита.

Коммит содержит снимок изменений, автора, дату, сообщение и связь с предыдущими коммитами. Благодаря этому проект можно анализировать во времени: видеть, когда появилась новая функция, кто исправил ошибку, какие файлы были изменены и к какой версии можно вернуться при необходимости.

Основные понятия Git включают репозиторий, рабочую директорию, индекс, коммит, ветку, слияние, удаленный репозиторий и тег. Репозиторий хранит историю проекта. Рабочая директория содержит текущие файлы. Индекс используется для подготовки изменений к коммиту. Ветки позволяют вести параллельную разработку без повреждения основной версии проекта.

Для GitOps особенно важна работа с ветками. Обычно основная ветка main или master содержит стабильное состояние. Для новой функции создается отдельная ветка. После завершения разработки изменения отправляются в удаленный репозиторий и проходят review. Только после проверки и успешного pipeline изменения попадают в основную ветку.

Таблица 1 - Базовые команды Git и их роль в GitOps-процессе

Команда	**Назначение**	**Роль в GitOps**
git init	Создание локального репозитория	Начало версионирования проекта
git clone	Копирование удаленного репозитория	Получение единого источника проекта
git status	Просмотр состояния файлов	Контроль изменений перед коммитом
git add	Добавление изменений в индекс	Подготовка изменения к фиксации
git commit	Создание коммита	Фиксация изменения с сообщением
git branch	Работа с ветками	Изоляция разработки функций
git merge	Слияние веток	Интеграция проверенных

		изменений
git pull	Получение изменений	Синхронизация с удаленным репозиторием
git push	Отправка изменений	Запуск review и pipeline на платформе

Использование Git дисциплинирует разработку: изменения становятся небольшими, понятными и проверяемыми. В GitOps это принципиально важно, потому что автоматизация имеет смысл только тогда, когда все входные данные находятся в репозитории и могут быть воспроизведены.

3. GitHub: совместная разработка и автоматизация workflows

GitHub - одна из наиболее популярных платформ для размещения Git-репозиториев и организации совместной разработки. Она предоставляет инструменты для pull request, code review, issue tracking, wiki, releases, security scanning и автоматизации через GitHub Actions.

Ключевой механизм командной разработки в GitHub - pull request. Разработчик создает ветку, вносит изменения и предлагает объединить их с основной веткой. Другие участники команды просматривают код, оставляют комментарии, требуют исправлений или подтверждают готовность изменений. Такой подход предотвращает случайное попадание непроверенного кода в основную ветку.

GitHub Actions позволяет автоматизировать задачи жизненного цикла программного обеспечения. В официальной документации GitHub workflow определяется как настраиваемый автоматизированный процесс, который содержит jobs и запускается событиями репозитория, вручную или по расписанию [4]. Workflow-файлы хранятся в каталоге .github/workflows и описываются на YAML.

GitHub Actions хорошо подходит для проектов, где требуется быстро настроить автоматическую проверку pull request: запуск линтера, unit-тестов, сборки, публикации артефактов и деплоя. Для GitOps важно, что workflow находится в репозитории вместе с кодом. Это значит, что логика автоматизации также версионируется и проходит review.

Пример простого GitHub Actions workflow:

```
name: CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
```

```
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
      - name: Install dependencies
        run: npm install
      - name: Run tests
        run: npm test
      - name: Build project
        run: npm run build
```

В данном примере workflow запускается при push и pull request в ветку main. Сначала код извлекается из репозитория, затем устанавливаются зависимости, запускаются тесты и выполняется сборка. Если хотя бы один шаг завершится ошибкой, весь workflow будет считаться неуспешным, а команда увидит проблему до слияния изменений.

4. GitLab: единая платформа для репозиториев и CI/CD

GitLab - веб-платформа для управления проектами и репозиториями программного кода на основе Git. В учебном материале Skillbox GitLab описывается как сервис для хранения кода, управления версиями и совместной разработки программного обеспечения [5]. В отличие от подхода, где для задач, репозитория, CI/CD и артефактов используются разные сервисы, GitLab стремится объединить эти функции в одной платформе.

Командная работа в GitLab строится вокруг merge request. Разработчик создает ветку, отправляет изменения и открывает merge request в основную ветку. Команда проверяет код, обсуждает изменения, запускает pipeline и только после успешных проверок выполняет слияние.

GitLab CI/CD является одной из сильных сторон платформы. Официальная документация указывает, что pipelines являются фундаментальным компонентом GitLab CI/CD и настраиваются в файле .gitlab-ci.yml с использованием YAML-ключевых слов [3]. Pipeline может запускаться автоматически при push, создании merge request, по расписанию или вручную.

В GitLab pipeline состоит из jobs и stages. Jobs выполняют конкретные команды: сборку, тестирование, линтинг, упаковку, публикацию артефактов или деплой. Stages группируют jobs и задают порядок выполнения. Обычно pipeline содержит этапы build, test и deploy. Если job на раннем этапе завершился с ошибкой, последующий этап обычно не запускается.

Пример простого .gitlab-ci.yml:

```
stages:
  - build
  - test
  - deploy

build_job:
  stage: build
  script:
    - echo "Building project"
    - npm install
```

```
    - npm run build
  artifacts:
    paths:
      - dist/

test_job:
  stage: test
  script:
    - echo "Running tests"
    - npm test

deploy_job:
  stage: deploy
  script:
    - echo "Deploying application"
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
      when: manual
```

В статье Хабр об основах написания GitLab Pipeline подчеркивается, что файл .gitlab-ci.yml обычно располагается в корне репозитория, а stages, jobs, script и rules являются ключевыми элементами конфигурации [6]. Во второй части статьи отдельно рассматриваются триггеры и условия запуска pipeline: push, merge request, расписание, ручной запуск и внешние события [7].

Для GitOps GitLab удобен тем, что код, конфигурация pipeline, артефакты, контейнерный registry, переменные CI/CD, среда развертывания и merge request могут находиться в одном месте. Это снижает организационную сложность и делает процесс более прозрачным для учебных и небольших команд.

5. Сравнение GitHub и GitLab в контексте GitOps

GitHub и GitLab решают похожие задачи, но делают акцент на разных аспектах. GitHub исторически воспринимается как крупнейшая платформа для open-source и collaborative development. GitLab чаще позиционируется как более цельная DevOps-платформа, в которой репозитории, CI/CD, артефакты, контейнерный registry и управление проектом тесно связаны.

Таблица 2 - Сравнение GitHub и GitLab

Критерий	GitHub	GitLab	Значение для GitOps
Основной объект review	Pull Request	Merge Request	Контроль изменений перед слиянием
CI/CD-конфигурация	.github/workflows/*.yml	.gitlab-ci.yml	Pipeline хранится в Git
Структура автоматизации	Workflow -> jobs -> steps	Pipeline -> stages -> jobs	Формализация сборки и тестирования
Запуск автоматизации	События, расписание, manual	Push, MR, schedule, manual, triggers	Автоматическая реакция на изменения
Инфраструктура runner	GitHub-hosted и self-hosted runners	GitLab runners	Выполнение jobs в контролируемой среде
Удобство для open-source	Очень высокое	Высокое	Публичная разработка и вклад сообщества
Единая DevOps-платформа	Через экосистему GitHub	Сильная интеграция внутри GitLab	Меньше внешних связок в проекте
Безопасность	Branch protection, secrets, code scanning	Protected branches, CI/CD variables, approvals	Контроль доступа к критическим изменениям

Для небольшой команды выбор между GitHub и GitLab зависит от требований проекта. Если важна публичность, большое open-source-сообщество и простая интеграция с внешними инструментами, GitHub может быть удобнее. Если требуется единая платформа с сильной встроенной CI/CD-логикой, управлением артефактами и более централизованной DevOps-моделью, GitLab часто оказывается предпочтительным.

В учебном контексте важно понимать не столько конкуренцию платформ, сколько общую логику: обе платформы используют Git, позволяют работать через ветки и review, хранят pipeline-конфигурации в репозитории и помогают автоматизировать сборку, тестирование и доставку изменений.

6. Практическая организация GitOps-процесса

Практический GitOps-процесс можно представить как цепочку действий, где каждое изменение проходит через Git и автоматические проверки. Такой процесс применим как к учебному веб-приложению, так и к промышленному сервису.

Типовой порядок работы:

- разработчик получает актуальную версию репозитория командой git pull;
- создается отдельная ветка для задачи, например feature/login-page;
- вносятся изменения в код, тесты или конфигурацию;
- изменения фиксируются через git add и git commit;
- ветка отправляется в GitHub или GitLab через git push;
- создается pull request или merge request;
- автоматически запускается pipeline: сборка, тесты, линтинг, проверка безопасности;
- участники команды выполняют review;
- после успешных проверок изменения сливаются в main;
- из основной ветки запускается доставка или развертывание.

Для GitOps особенно важно, чтобы pipeline не был устной инструкцией или набором команд на компьютере одного разработчика. Он должен быть описан в репозитории. Тогда любой участник команды видит, как именно собирается и проверяется проект, а изменения в pipeline также проходят историю Git.

Таблица 3 - Этапы GitOps-процесса и ответственные инструменты

Этап	Действие	Инструмент	Результат
Code	Создание ветки и коммитов	Git	Зафиксированная история изменений
Review	Обсуждение и проверка кода	GitHub/GitLab	Одобренный pull/merge request
Build	Сборка проекта	GitHub Actions/GitLab CI	Артефакт сборки
Test	Unit, integration, e2e тесты	Jest, JUnit, Playwright	Подтверждение качества
Security	Проверка зависимостей и секретов	CI/CD tools	Снижение рисков
Release	Создание версии и тега	Git tags, Releases	Идентифицируемая версия

Deploy	Доставка в среду выполнения	CI/CD, GitOps CD	Обновленное приложение
Monitor	Контроль состояния	Prometheus, Grafana, GitLab metrics	Обратная связь

Для учебной работы достаточно продемонстрировать понимание того, что GitOps строится вокруг малых изменений, автоматических проверок и управляемого слияния. В промышленной практике к этому добавляются защищенные ветки, обязательные approvals, секреты, отдельные окружения, теги версий, контейнеры и системы мониторинга.

7. Безопасность, контроль качества и типичные ошибки

GitOps повышает надежность разработки, но только при правильной организации процесса. Если команда хранит пароли в репозитории, игнорирует review или отключает тесты ради скорости, преимущества GitOps быстро теряются. Поэтому безопасность и контроль качества должны быть встроены в pipeline.

К базовым мерам безопасности относятся protected branches, обязательный review, запрет прямого push в main, хранение секретов в специальных настройках платформы, ограничение прав доступа и регулярный аудит зависимостей. В GitHub для этого используются branch protection rules, secrets и security tools. В GitLab - protected branches, approvals, CI/CD variables и настройки runner.

Типичные ошибки начинающих команд: слишком большие merge request, отсутствие понятных сообщений коммитов, хранение секретов в коде, запуск тестов только перед релизом, отсутствие линтера, незафиксированные ручные изменения инфраструктуры, отсутствие стратегии отката. Все эти проблемы противоречат идее GitOps, потому что делают процесс непрозрачным и невоспроизводимым.

Контроль качества должен включать несколько уровней: синтаксическую проверку, форматирование, unit-тесты, интеграционные тесты, проверку сборки, анализ зависимостей и review. В материалах по CI/CD отдельно выделены инструменты для сборки и тестирования, включая Gradle, Bazel, Webpack, Jest, JUnit и Jenkins [10]. Это показывает, что GitOps не существует отдельно от CI/CD: Git фиксирует изменения, а pipeline проверяет их техническое качество.

Таблица 4 - Риски GitOps и способы их снижения

Риск	**Причина**	**Профилактика**
Секреты попали в Git	Пароли записаны в YAML или код	Использовать secrets/CI variables
Сломанная основная ветка	Слияние без тестов	Protected branch и обязательный pipeline
Конфликты кода	Долгая работа в одной ветке	Часто синхронизироваться с main

Непонятная история	Плохие сообщения коммитов	Единый стандарт commit message
Ручные изменения в production	Обход GitOps-процесса	Все изменения только через Git
Pipeline не запускается	Ошибка YAML или runner	Валидация конфигурации и проверка runner

Заключение

В ходе выполнения обзорной реферативной работы были рассмотрены технологии Git, GitHub и GitLab в контексте GitOps. Было установлено, что GitOps является современным подходом к организации разработки и эксплуатации, при котором Git-репозиторий выступает центральным источником истины для кода, конфигураций и автоматизированных процессов.

Git обеспечивает надежное хранение истории изменений, работу с ветками, коммитами и удаленными репозиториями. GitHub и GitLab расширяют возможности Git, предоставляя командам инструменты совместной разработки, review, управления задачами, CI/CD и контроля доступа. GitHub делает сильный акцент на pull request и GitHub Actions, а GitLab предлагает единую платформу с merge request и встроенным GitLab CI/CD.

Отдельно были рассмотрены pipeline-конфигурации GitHub Actions и GitLab CI/CD. Было показано, что автоматизация сборки, тестирования и доставки должна храниться в репозитории, потому что именно это делает процесс воспроизводимым, проверяемым и управляемым. GitOps позволяет снизить риски ручных ошибок, ускорить получение обратной связи и повысить качество программного обеспечения.

Таким образом, технологии Git, GitHub и GitLab являются фундаментом современной разработки. Их грамотное применение позволяет организовать прозрачный процесс, в котором каждое изменение фиксируется, проверяется, обсуждается и только после этого доставляется в основную кодовую базу или среду выполнения. Цель работы достигнута, так как раскрыты основные понятия GitOps, изучены ключевые инструменты и показана их практическая связь с CI/CD.

Список использованных источников

1. Git Documentation. Официальная документация Git. https://git-scm.com/docs/git

2. Pro Git Book. About Version Control. https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

3. GitLab Docs. CI/CD pipelines. https://docs.gitlab.com/ci/pipelines/

4. GitHub Docs. Workflows. https://docs.github.com/en/actions/concepts/workflows-and-actions/workflows

5. Skillbox Media. GitLab: что это такое и как им пользоваться. https://skillbox.ru/media/code/gitlab-chto-eto-takoe-i-kak-im-polzovatsya/

6. Хабр. GitLab: Основы написания Pipeline 1/3. https://habr.com/ru/articles/980478/

7. Хабр. GitLab: Основы написания Pipeline 2/3. https://habr.com/ru/articles/981658/

8. OpenGitOps. About OpenGitOps. https://opengitops.dev/about/

9. GitHub Actions documentation. https://docs.github.com/actions

10. Материалы к лабораторной работе: CICD (1).pdf. Файл, предоставленный преподавателем.

11. Jenkins documentation. https://www.jenkins.io/doc/

12. GitLab Docs. CI/CD YAML syntax reference. https://docs.gitlab.com/ci/yaml/

13. GitHub Docs. Workflow syntax for GitHub Actions. https://docs.github.com/actions/using-workflows/workflow-syntax-for-github-actions

14. GitHub Features. GitHub Actions. https://github.com/features/actions